Exploration Should Act at the Group Level in RL LLM Fine-Tuning

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

What role should exploration take in GRPO-style reasoning RL? Existing exploration mechanisms act either by (1) scaling entire prompts with uncertainty estimates or by (2) increasing token-level stochasticity. Yet, under GRPO, RL learns from groups: for each prompt, the policy samples several completions and the update is driven by comparisons inside that prompt-local set. To align exploration with model update, we introduce *xDr.GRPO*, a maximum-entropy variant of Dr.GRPO that places exploration directly on the sampled candidate distribution. For each prompt, xDr.GRPO constructs a target distribution over completions by combining reward-induced group fit with candidate-level Shannon entropy. The resulting target has a closed form and gives a clean way to separate prompt-level, token-level, and candidate-level regularization. We evaluate this design with a matched multi-seed regression analysis on two synthetic exact multi-answer benchmarks — Countdown arithmetic and graph-coloring completion — in which every prompt admits several verifiably distinct valid answers, using aggregate accuracy and answer-mode coverage as descriptive evidence and regression coefficients to estimate effect size and uncertainty across task domains, training seeds, and evaluation seeds.

1 Introduction

Reinforcement learning (RL) has become a central component in the post-training for large language models (LLMs), especially for reasoning tasks where the desired behavior cannot be fully specified by supervised imitation alone [28, 37, 12]. Verifiable domains such as mathematics and code are particularly natural testbeds: candidate solutions can often be checked automatically, and recent work increasingly uses outcome or process feedback to improve reasoning [14, 4, 20, 21, 18]. In these settings, learning depends not only on rewarding successful answers but also on whether training produces useful comparisons among alternative reasoning paths. The central question is therefore: how should an RL fine-tuning algorithm encourage a model to generate a useful group of reasoning candidates at each training step?

This question holds particular significance for Group Relative Policy Optimization (GRPO). GRPO-style methods sample multiple completions for the same prompt, compute rewards within that prompt-local group, and update the policy from the resulting relative signal [37, 24]. The model does not learn from a single completion in isolation but rather from the structure of a sampled set. Exploration is therefore critical in two respects: it determines which reasoning paths are available for comparison, and it affects how informative the resulting group-relative update can be. Prior work on self-consistency and related sampling methods shows that diverse reasoning paths can materially improve reasoning performance, suggesting that useful alternatives matter even when only one final answer is selected [44, 22].

A mismatch arises because the supervision in GRPO is groupwise, while most exploration mechanisms are not. Some methods act at the prompt level, for example by using semantic uncertainty to scale the update for an entire question [3, 9]. Others act at the token level, for example by adding a next-token entropy bonus to the loss [46, 27] or by entropy-regularized token-level objectives for language agents [45]. Both can be useful, but neither directly regularizes the prompt-local distribution over sampled completions, which is the object GRPO uses to assign credit. Raising sampling temperature can increase surface-level variation, but it does not ensure that the group contains meaningfully different or useful reasoning paths. This strategy can just as easily create redundant or incorrect strategies, noisy off-policy completions, or higher-variance updates.

We propose **xDr.GRPO**, a candidate-level maximum-entropy formulation built on the Dr.GRPO backbone [24]. The core idea is simple: if GRPO learns from a group, exploration should be expressed on that same group. For each prompt, xDr.GRPO constructs a distribution over the sampled completions. This distribution balances two forces: higher probability for higher-reward candidates and Shannon entropy over the candidate set (an optional reference tilt toward the base model is analyzed in Appendix B.1 but not used in our experiments). The resulting weights have a closed form, are computed once per rollout batch and frozen for the update exactly as advantages are, and preserve compatibility with the standard prompt-group rollout pipeline: only the aggregation weights of the Dr.GRPO surrogate change.

Our evaluation is designed to match this conceptual claim. Rather than relying solely on pooled accuracy tables or best-checkpoint summaries, we evaluate the method with a regression-based analysis over matched runs. Each observation corresponds to a method, training seed, evaluation seed, task domain, prompt, and sampled completion or prompt-level aggregate. The coefficient on the xDr.GRPO indicator estimates the accuracy and coverage lift relative to Dr.GRPO after controlling for task-domain and seed effects. This approach allows us to report not only whether xDr.GRPO improves aggregate performance, but also the size of the estimated effect and the remaining uncertainty.

Our contributions are as follows. **(i)** We identify a structural mismatch in GRPO-style exploration: the learning signal is formed over sampled completion groups, while common regularizers act at the prompt or token level. **(ii)** We propose a groupwise maximum-entropy objective that regularizes the prompt-local candidate distribution and derive its closed-form target. **(iii)** We instantiate the objective as xDr.GRPO, a Dr.GRPO-based policy optimization method that can be compared against scalar group-relative updates, semantic-entropy scaling, and token-level entropy regularization under a shared backbone. **(iv)** We give the objective a KL-proximal interpretation and provide robustness guarantees. **(v)** We evaluate the method with a matched multi-seed regression protocol on two synthetic exact multi-answer benchmarks — Countdown arithmetic and graph-coloring completion — whose prompts each admit several verifiably distinct valid answers, making effect size and statistical uncertainty explicit.

2 Related Work

KL-regularized and maximum-entropy policy optimization. A broad line of work views policy optimization through KL-regularized or mirror-descent updates. Classical examples include REPS, MPO/V-MPO, MDPO, TRPO, and PPO, which use relative-entropy or trust-region constraints to stabilize policy improvement [33, 1, 39, 41, 35, 36]. This perspective has been formalized in regularized RL and policy mirror-descent frameworks [11, 47, 43]. Maximum-entropy RL adds an explicit entropy term to the control objective, yielding softened or Boltzmann-style solutions and connecting RL to control-as-inference and KL-control [13, 19, 49, 40]. Two closer relatives deserve note. Exponential advantage weighting of the form $w \propto \exp(A/\tau)$ is the E-step of reward-weighted and advantage-weighted regression [32, 31], and the log-sum-exp aggregation our objective reduces to is the *entropic risk measure* of risk-sensitive control [16, 26]. Our formulation builds on this tradition, but applies the exponential weights and the entropy term directly to a *finite set of sampled candidates* within a prompt group, rather than to the full token-level action distribution or to a replay buffer of trajectories.

Preference optimization and groupwise supervision for language models. Recent post-training methods for language models often optimize KL-regularized objectives induced by preferences or rankings. DPO, SimPO, ORPO, and KTO recast alignment as preference optimization without relying on standard on-policy RL pipelines [34, 25, 15, 8]. In parallel, group methods such as

LiPO and PRO emphasize supervision over sets of candidate responses rather than isolated pairwise comparisons [23, 38]. xDr.GRPO is complementary to these approaches: it retains an on-policy GRPO-style training loop, but introduces a prompt-local target distribution over sampled completions and regularizes that distribution directly.

GRPO and its variants. GRPO is a critic-free, PPO-style post-training method that samples a group of completions, forms group-relative learning signals, and broadcasts them across tokens within each completion [37]. Dr.GRPO revisits this update and motivates a corrected backbone that reduces optimization pathologies in R1-Zero-style training [24]. Several recent variants then modify where exploration or uncertainty enters the GRPO pipeline. SEED-GRPO uses semantic entropy to scale updates at the prompt level [3, 9], Info-GRPO introduces diversity through latent conditioning [2, 42], and token-level entropy interventions range from the classic loss-side entropy bonus [46, 27] to soft-Bellman objectives that carry a KL-to-reference regularizer inside the token-level backup [45]. Our method differs in where it intervenes: it regularizes the *distribution over sampled candidates* itself, yielding a closed-form prompt-local target and a direct candidate-level view of exploration in GRPO-style reasoning RL.

3 Where Exploration Enters the Objective

GRPO-style methods learn from a *group*. For each prompt, the policy samples several completions, scores them, and forms a within-prompt comparison. Our claim is that this group is also the natural place for exploration to act. We first observe that Dr.GRPO already contains an aggregation step, and that this step is a hidden, unexamined design choice.

Common prompt-group setup. For each prompt x, the rollout policy μ samples G completions $\{y_{x,1}, \dots, y_{x,G}\}$ with scalar rewards $r_{x,i}$, group mean $\bar{r}_x = \frac{1}{G}\sum_i r_{x,i}$, and centered advantages $A_{x,i} = r_{x,i} - \bar{r}_x$. Let $m_{x,i,t}$ be the completion-token mask, $c_{x,i,t}$ the tokenwise importance ratio between π_θ and μ, and $\tilde{c}_{x,i,t}$ its clipped version (Appendix A.1). Define the *per-candidate Dr.GRPO utility*

$$\bar{U}_{x,i}(\theta) \;:=\; \frac{1}{T_{\max}} \sum_t m_{x,i,t}\, \min\!\big(c_{x,i,t}A_{x,i},\; \tilde{c}_{x,i,t}A_{x,i}\big), \tag{1}$$

where the constant $T_{\max}$ normalizer is exactly Dr.GRPO's length-bias correction [24]. Since $|A_{x,i}| \le 1$ for binary verifiable rewards and the mask has at most $T_{\max}$ active positions, $|\bar{U}_{x,i}| \le 1$ up to the clip range.

Dr.GRPO is a risk-neutral aggregator. Written this way, the Dr.GRPO objective is, up to a constant,

$$\mathcal{J}_{\mathrm{Dr.GRPO}}(\theta) = \frac{1}{|\mathcal{A}|}\sum_{x\in\mathcal{A}} \Big\langle \mathbf{u},\; \bar{U}_x(\theta) \Big\rangle, \qquad \mathbf{u} = \big(\tfrac{1}{G}, \dots, \tfrac{1}{G}\big) \in \Delta^G. \tag{2}$$

That is: Dr.GRPO builds a group, computes a utility for each candidate, and then *averages them uniformly*. The group-relative signal lives entirely inside $A_{x,i}$; the aggregation across candidates is a plain mean. Nothing in the derivation of Dr.GRPO requires this. The uniform weights are an implicit choice, and they are the choice that treats every sampled candidate as equally worth learning from.

xDr.GRPO: optimistic aggregation over the candidate group. xDr.GRPO replaces $\mathbf{u}$ by a prompt-local distribution $w_x \in \Delta^G$ chosen variationally. For $\tau > 0$ and $\lambda \ge 0$,

$$\mathcal{G}_x(\theta) \;:=\; \max_{w\in\Delta^G}\Big\{ \underbrace{\langle w, \bar{U}_x(\theta)\rangle}_{\text{same Dr.GRPO utilities}} \;+\; \underbrace{\tau H(w)}_{\text{candidate-level entropy}} \;-\; \underbrace{\lambda\, \mathrm{KL}\big(w \,\|\, \pi_x^{\mathrm{ref}}\big)}_{\text{reference tilt}} \Big\}, \tag{3}$$

where $\pi_x^{\mathrm{ref}} \in \Delta^G$ is a reference-induced distribution over the same sampled set (Appendix B.1), and the training objective is $\mathcal{J}_{\mathrm{xDr.GRPO}}(\theta) = \frac{1}{|\mathcal{A}|}\sum_{x\in\mathcal{A}} \mathcal{G}_x(\theta)$. The maximizer is a tilted softmax over utilities,

$$w^\star_{x,i}(\theta) = \frac{\exp\!\big(\bar{U}_{x,i}(\theta)/(\tau+\lambda)\big)\,\big(\pi^{\mathrm{ref}}_{x,i}\big)^{\lambda/(\tau+\lambda)}}{\sum_{j=1}^{G} \exp\!\big(\bar{U}_{x,j}(\theta)/(\tau+\lambda)\big)\,\big(\pi^{\mathrm{ref}}_{x,j}\big)^{\lambda/(\tau+\lambda)}}, \tag{4}$$

and eliminating w leaves a log-partition (soft-max) objective

$$\mathcal{G}_x(\theta) = (\tau+\lambda)\,\log\sum_{i=1}^{G}\exp\!\Big(\tfrac{\bar U_{x,i}(\theta)}{\tau+\lambda}\Big)\big(\pi^{\mathrm{ref}}_{x,i}\big)^{\lambda/(\tau+\lambda)}. \tag{5}$$

Proofs are in Appendix B. Crucially, by the envelope theorem the gradient is the *same per-candidate Dr.GRPO gradient, reweighted*:

$$\nabla_\theta \mathcal{G}_x(\theta) = \sum_{i=1}^{G} w^\star_{x,i}(\theta)\,\nabla_\theta \bar U_{x,i}(\theta), \tag{6}$$

with $w^\star$ entering as a constant: the implementation computes the weights once per rollout batch at the rollout policy — where the clipped utility reduces to the closed form $A_{x,i}T_{x,i}/T_{\max}$ — and freezes them for the update, exactly as the advantages are frozen (Remark A.1). The rollout pipeline, reward function, clipping, and tokenwise surrogate are untouched. *Only the aggregation weights change.*

Why this is exploration, and why Dr.GRPO has none. Equation (5) is a tempered log-sum-exp: the *entropic risk measure* of the candidate utilities [16, 26], with $\tau+\lambda$ as inverse risk-sensitivity. Its second-order expansion (Proposition B.4) makes the mechanism explicit:

$$\mathcal{G}_x(\theta) = \underbrace{\tau\log G}_{\text{constant}} + \underbrace{\frac{1}{G}\sum_{i=1}^{G}\bar U_{x,i}(\theta)}_{\textbf{Dr.GRPO}} + \underbrace{\frac{1}{2\tau}\,\mathrm{Var}_i\big(\bar U_{x,i}(\theta)\big)}_{\textbf{group-level exploration bonus}} + O(\tau^{-2}) \qquad (\lambda=0). \tag{7}$$

Three consequences follow, and they replace the informal "entropy adds stochasticity" story used by prior work.

(i) Dr.GRPO is the risk-neutral member of this family. As $\tau\to\infty$, $w^\star\to\mathbf{u}$ and $\nabla_\theta\mathcal{G}_x\to\nabla_\theta\mathcal{J}_{\mathrm{Dr.GRPO},x}$ (Proposition B.5). The baseline is not "xDr.GRPO without exploration"; it is *xDr.GRPO at zero risk-sensitivity*. Exploration is therefore not an additive gadget bolted onto Dr.GRPO — it is a parameter Dr.GRPO silently pins at a corner.

(ii) Finite τ is optimism over the group. By Jensen, $\mathcal{G}_x - \tau\log G \geq \mathcal{J}_{\mathrm{Dr.GRPO},x}$, with the gap equal to the within-group *dispersion* of candidate utilities. xDr.GRPO thus prefers parameters that keep the sampled group spread out in utility — that is, prompts and policies where the group still disagrees. This is optimism in the face of uncertainty, expressed on the object GRPO uses for credit assignment, rather than on the prompt (SEED-Dr.GRPO) or the next-token distribution (Token-MaxEnt Dr.GRPO).

(iii) The mechanism is asymmetric gradient attenuation. Because $w^\star_{x,i}\propto\exp(\bar U_{x,i}/\tau)$ and $\bar U_{x,i}$ is increasing in $A_{x,i}$, negative-advantage candidates receive *exponentially downweighted* gradients. How such attenuation interacts with policy-entropy dynamics under verifiable rewards is genuinely contested: entropy collapse has been traced to high-covariance updates on confident *positive*-advantage tokens [5], and sequence-level negative reinforcement alone can even preserve entropy [48], while negative gradients applied to *low-probability* tokens are a documented entropy-collapsing and likelihood-displacing force whose selective attenuation helps [30, 17, 6]. Our reweighting acts at the candidate level rather than the token level, so we treat its net entropy effect as an empirical question — Figure 1 measures it directly through the effective number of active rollouts and the incorrect-mass share, logged identically for every arm.

The family, end to end. The temperature interpolates between two known regimes:

$\tau\to 0$	$w^\star\to\mathbf{1}\{i=\arg\max_i \bar U_{x,i}\}$	best-of-G distillation (RAFT-style, 7)
$0<\tau<\infty$	tilted softmax over utilities	**xDr.GRPO**
$\tau\to\infty$	$w^\star\to\mathbf{u}$	**Dr.GRPO**

Our empirical claim is that the optimum lies in the *interior*: rejection-sampling-style concentration discards the comparative signal that makes GRPO work, while uniform aggregation spends a fixed share of every update on negative candidates — whether that degrades the sampled group is exactly what the diagnostics of Figure 1 test. Section 5 tests this with a sweep over τ in which Dr.GRPO appears as the $\tau=\infty$ endpoint of the same family, run through the same launcher with the reweighting disabled (Remark A.1).

Degenerate groups need no special handling. If all sampled rewards are equal, then $A_{x,i} = 0$ for all i, hence $\bar{U}_{x,i} = 0$, $w_x^\star$ is uniform (or reference-tilted), and $\nabla_\theta \mathcal{G}_x = 0$. Uninformative prompts contribute no gradient automatically, matching Dr.GRPO's behavior; no prompt-skipping heuristic is required.

4 Compared Methods

We organize the comparison around where exploration enters an otherwise shared GRPO-style training pipeline. The rollout interface, reward function, base model, reference model, and optimization budget are held fixed. The methods differ only in the ways they transform a prompt-local group of sampled completions into an update.

Dr.GRPO. Dr.GRPO is the scalar-weight baseline. It centers rewards within each prompt group, assigns each sampled completion a scalar advantage, and applies the clipped Dr.GRPO surrogate [37, 24]. It therefore uses group-relative information, but collapses the group before optimization.

xDr.GRPO. xDr.GRPO is the candidate-level variant. It constructs a prompt-local weight distribution over the sampled completions and reweights each candidate's Dr.GRPO update by it. This method directly regularizes the object being compared inside GRPO: the set of sampled candidates for a prompt.

Token-MaxEnt Dr.GRPO. Token-level MaxEnt keeps the Dr.GRPO backbone but adds the classic entropy bonus to the next-token objective [46, 27], encouraging locally higher-entropy token distributions during optimization; entropy-regularized token-level objectives for language agents pursue the same intuition through a soft-Bellman formulation [45]. This approach tests whether token-level stochasticity alone can provide the same benefit as candidate-level exploration.

SEED-Dr.GRPO. SEED-style Dr.GRPO preserves the clipped Dr.GRPO update but rescales prompt updates using semantic uncertainty, following semantic-entropy approaches that estimate uncertainty over answer meaning rather than over individual token choices [3, 9]. This technique thus tests a prompt-level intervention.

Together, these methods form a clean axis: no extra exploration beyond rollout sampling (Dr.GRPO), prompt-level uncertainty scaling (SEED-Dr.GRPO), token-level entropy regularization (Token-MaxEnt Dr.GRPO), and candidate-level distribution matching (xDr.GRPO). The main paired regression analysis focuses on the completed Dr.GRPO versus xDr.GRPO runs; the same regression specification extends directly to the full quartet by replacing the binary method indicator with method fixed effects.

5 Experimental Design

We evaluate xDr.GRPO in verifiable reasoning tasks with *multiple valid solutions per prompt*. The testbed consists of two synthetic exact multi-answer domains, generated by exhaustive enumeration so that the full set of valid answers for every prompt is known.

Countdown arithmetic. Each prompt presents three distinct integers drawn from $\{2, \dots, 12\}$ and an integer target, and asks the model to return a single arithmetic expression, inside `\boxed{}`, that reaches the target using each given number exactly once with $+$, $-$, $\times$, $/$, and parentheses [10, 29]. All reachable expressions are enumerated exactly (every permutation of the numbers, every binary expression tree, rational arithmetic), and each expression is reduced to a canonical form in which commutative operands are sorted while operator identity is preserved. A prompt's *answer modes* are its distinct canonical solution expressions: for example, the numbers $[2, 3, 4]$ reach the target 10 through the two distinct modes $2 \times 3 + 4$ and $3 \times 4 - 2$.

Graph-coloring completion. Each prompt presents a small undirected graph (vertices 1 through n), its edge list, and a partial coloring with hidden vertices, and asks the model to return colors from $\{1, 2, 3\}$ for the hidden vertices, boxed, such that no edge has equal colors at both endpoints. All valid colorings consistent with the partial assignment are enumerated exactly; a prompt's answer modes are its distinct valid completions.

In both domains, prompts are filtered so that every training and evaluation item admits several modes (between 2 and 8 for Countdown), which makes the pair a direct probe of whether a training objective finds *more of the valid answer set* rather than one answer more reliably. The problem scale is calibrated so that the base policy earns reward often enough to bootstrap: three-number Countdown gives an untrained small model a workable success rate, whereas four-number variants yield almost no rewarded rollouts and hence no learning signal. The released configurations use 384 training prompts and 128 held-out evaluation prompts for Countdown (192/96 for graph coloring), generated deterministically from fixed seeds, plus optional single-mode control splits in which each prompt has exactly one valid answer. Evaluation prompts are excluded from the training pool by problem identity (numbers and target for Countdown; graph, edge set, and partial coloring for graph coloring), so no evaluation item is ever trained on. In the released Countdown configuration the evaluation split's mode counts concentrate at five modes per prompt (median 5, range 2–8) with targets in $[1, 36]$. The primary paired comparison is Dr.GRPO versus xDr.GRPO, fine-tuning Qwen2.5-0.5B-Instruct policies with $G = 16$ rollouts per prompt during training. For each method, we train three independent seeds per domain and evaluate each trained checkpoint with three evaluation seeds; evaluation draws $K = 8$ sampled completions per prompt at temperature 1.0 (independent of the training group size) plus one greedy completion, always from the final saved checkpoint — a rule fixed before any evaluation was run.

All methods use the same base model, prompt format, reward extraction logic, rollout budget, and evaluation harness. Correctness is verifiable and binary: the boxed expression is normalized, parsed, and evaluated with exact rational arithmetic, and receives reward 1 if and only if it equals the target and uses each given number exactly once. Mode identity is never part of the training reward — the trainer sees correctness only — so any difference in how many distinct modes a method discovers is attributable to the training objective rather than to mode-aware supervision.

5.1 Metrics

We compute five descriptive metrics. Greedy `pass@1` measures whether the deterministic completion is correct. Sampled `pass@8` measures whether at least one of eight sampled completions is correct for a prompt. Sampled `mean@8` measures the average correctness rate across the eight sampled completions. Because every prompt admits several distinct valid modes, we additionally track `distinct@8`, the number of distinct canonical answer modes among the correct sampled completions, and `coverage@8`, defined as `distinct@8` divided by the prompt's total number of valid modes. The coverage metrics measure exactly what candidate-level exploration is supposed to buy: discovering more of the valid answer set, not only finding one answer more often. These aggregate metrics are useful summaries, but they do not by themselves quantify uncertainty across difficulty strata, training seeds, and evaluation seeds. For that reason, our primary statistical comparison is regression-based.

5.2 Regression-Based Evaluation

For each method m, training seed s, evaluation seed e, task domain b (Countdown or graph coloring), prompt p, and sampled completion k, let

$$C_{m,s,e,b,p,k} \in \{0, 1\}$$

indicate whether the normalized boxed answer is correct. We form prompt-level outcomes from these completion-level indicators:

$$Y^{\texttt{pass@8}}_{m,s,e,b,p} = \mathbf{1}\left\{ \max_{1\le k\le 8} C_{m,s,e,b,p,k} = 1 \right\}, \qquad Y^{\texttt{mean@8}}_{m,s,e,b,p} = \frac{1}{8}\sum_{k=1}^{8} C_{m,s,e,b,p,k}.$$

For greedy `pass@1`, $Y^{\texttt{pass@1}}_{m,s,e,b,p}$ is the corresponding greedy correctness indicator. For coverage, let $d_{m,s,e,b,p}$ be the number of distinct canonical answer modes among the correct sampled completions and $M_{b,p}$ the prompt's known number of valid modes; then

$$Y^{\texttt{coverage@8}}_{m,s,e,b,p} = \frac{d_{m,s,e,b,p}}{M_{b,p}} \in [0, 1].$$

Quantity	Role in the regression analysis
Method indicator $\mathbf{1}\{m = \text{xDr.GRPO}\}$	Main treatment variable. Its coefficient estimates the accuracy and coverage lift of candidate-level exploration relative to Dr.GRPO.
Task-domain fixed effects δ_b	Control for baseline difficulty differences between Countdown arithmetic and graph-coloring completion.
Training-seed fixed effects η_s	Control for variation induced by independent fine-tuning runs.
Evaluation-seed fixed effects ζ_e	Control for sampling variation during stochastic evaluation.
Prompt-clustered standard errors	Account for repeated observations of the same problem across methods and seeds.

Table 1: **Regression-based evaluation protocol.** Aggregate `pass@1`, `pass@8`, `mean@8`, `distinct@8`, and `coverage@8` are reported descriptively, while the coefficient on the method indicator provides the primary effect-size and uncertainty estimate.

For each outcome Y, we estimate the paired linear probability model

$$Y_{m,s,e,b,p} = \alpha + \gamma\,\mathbf{1}\{m = \text{xDr.GRPO}\} + \delta_b + \eta_s + \zeta_e + \varepsilon_{m,s,e,b,p}. \tag{8}$$

Here δ_b are task-domain fixed effects, η_s are training-seed fixed effects, and ζ_e are evaluation-seed fixed effects. The coefficient γ is the estimated effect of xDr.GRPO relative to Dr.GRPO in accuracy points, after controlling for the matched evaluation design. We report $\hat{\gamma}$, its standard error, a confidence interval, and a p-value. Standard errors are clustered by evaluation prompt when prompt identities are shared across methods. We additionally check robustness to clustering by training run when reporting pooled results across task domains. Table 1 summarizes the role of each regression component.

This regression analysis is the paper's main answer to the question "is the observed improvement statistically distinguishable from seed and dataset variation?" The raw accuracy tables show the size and direction of the observed differences. Equation 8 tests whether those differences persist after accounting for the repeated-seed evaluation structure.

5.3 Power and Sample Size Planning

The study is powered around the regression coefficient γ in Equation 3, which estimates the absolute accuracy lift from replacing Dr.GRPO with xDr.GRPO. Power is assessed at the prompt level rather than the completion level, since completions from the same prompt and repeated evaluations of the same evaluation item are not independent. In the main paired comparison, each method is trained with three independent fine-tuning seeds and each checkpoint is evaluated with three stochastic evaluation seeds, yielding up to $2 \times 3 \times 3$ method-run evaluations per prompt across the two task domains. Before interpreting null effects as evidence of no improvement, we will estimate the minimum detectable effect for each outcome using the prompt-clustered standard error from the planned regression or, equivalently, a cluster bootstrap that resamples prompts and training runs.

For a two-sided test with $\alpha = 0.05$ and 80% power, the approximate minimum detectable effect is

$$\text{MDE} \approx \left(z_{1-\alpha/2} + z_{0.80}\right)\widehat{\text{SE}}(\hat{\gamma}) = (1.96 + 0.84)\widehat{\text{SE}}(\hat{\gamma}) \approx 2.8\,\widehat{\text{SE}}(\hat{\gamma}).$$

Thus, if the clustered standard error for $\hat{\gamma}$ is 0.01, the design can detect an improvement of roughly 2.8 percentage points; if it is 0.02, the detectable improvement is roughly 5.6 percentage points. We will treat an absolute gain of 3 to 5 points on `pass@8`, `mean@8`, or `coverage@8` as the smallest practically meaningful effect. If the estimated MDE is larger than this range, the experiment will be reported as underpowered and interpreted descriptively rather than as strong evidence for or against xDr.GRPO. With 128 evaluation prompts per domain and binary prompt-level outcomes, realistic prompt-clustered standard errors fall in the 0.013–0.027 range, corresponding to minimum detectable effects of roughly 4–8 accuracy points; effects smaller than that are beyond this design's resolution and will be treated as such.

5.4 Interpreting the Current Run

The current model scale is intentionally small, so we treat the experiment as a controlled test of the objective rather than a claim that group-level exploration already dominates at scale. A small positive aggregate difference can be meaningful only if it remains significant after the regression analysis above. Conversely, a non-significant coefficient would indicate that the present run does not yet separate the method effect from seed, dataset, and sampling noise.

The benchmark also makes the failure mode we target directly visible, which yields an *ordered* prediction rather than a single accuracy comparison. A policy can be perfectly accurate on these tasks while knowing only one way to be right: when $[2, 3, 4]$ reaches 10 through both $2 \times 3 + 4$ and $3 \times 4 - 2$, outcome reward is indifferent between a policy that can produce either mode and one that has collapsed onto a single expression, and the asymmetric negative-advantage gradients of uniform aggregation actively push toward the latter (Section 3). The prediction is therefore: Dr.GRPO and xDr.GRPO should be roughly comparable on `pass@1`, `pass@8`, and `mean@8`, while xDr.GRPO should retain strictly more of the valid answer set as measured by `distinct@8` and `coverage@8`. Coverage gains *with* accuracy losses, or accuracy gains *without* coverage gains, would both contradict the mechanism and will be reported as such. This framing keeps the empirical claim aligned with the mechanism: xDr.GRPO should improve learning when candidate-level exploration creates better within-prompt comparisons, and the regression estimates whether that effect is visible in the completed runs.

5.5 Threats to Validity and Expected Failure Modes

Several threats could limit the interpretation of the proposed comparison. First, although the verifier is exact — expressions are evaluated with rational arithmetic and colorings are checked against the enumerated valid set, so false positives are essentially ruled out — the strict answer-extraction gate can produce false negatives on malformed but semantically correct outputs. We will therefore inspect a sample of rejected completions from each domain and report robustness to stricter and more permissive extraction rules. Relatedly, what counts as a *distinct* mode is a canonicalization convention (commutative operands are merged; operator identity is preserved), and coverage results should be read relative to that convention. Second, the study could overstate the effect of xDr.GRPO if the two methods receive unequal hyperparameter tuning or if checkpoint selection is based on the same evaluation splits used for final reporting. To reduce this risk, all main runs will use a matched training budget, shared evaluation harness, and a pre-specified checkpoint-selection rule. Third, `pass@8` and `coverage@8` may reward diversity without necessarily reflecting better single-solution reasoning, while `mean@8` may penalize exploratory methods that produce one strong answer and several weak alternatives. We therefore interpret improvements jointly across `pass@1`, `pass@8`, `mean@8`, and `coverage@8`, and use the single-mode control split as a guardrail that single-answer performance is preserved. Finally, as the current experiment uses a relatively small 0.5B model on synthetic tasks whose full answer sets are enumerable, a positive effect should be interpreted as evidence that candidate-level exploration changes the learning signal under this controlled setting, not as a claim that xDr.GRPO will dominate at larger scales, on natural reasoning corpora, or under different reward models. In cases where the method improves sampled metrics but not greedy accuracy, or where gains appear only in one task domain, we will report this as a failure mode rather than as broad evidence of superior reasoning.

References

[1] Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Rémi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. In *International Conference on Learning Representations (ICLR)*, 2018. URL `https://openreview.net/forum?id=S1ANxQW0b`.

[2] Anonymous. Info-GRPO: Training reasoning models via correlation-aware exploration. OpenReview preprint (submitted to ICLR 2026), 2025. URL `https://openreview.net/forum?id=d5qElNtXS5`.

[3] Minghan Chen, Guikun Chen, Wenguan Wang, and Yi Yang. SEED-GRPO: Semantic entropy enhanced GRPO for uncertainty-aware policy optimization. arXiv preprint arXiv:2505.12346, 2025. URL `https://arxiv.org/abs/2505.12346`.

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John

Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[5] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. arXiv preprint arXiv:2505.22617, 2025. URL `https://arxiv.org/abs/2505.22617`.

[6] Wenlong Deng, Yi Ren, Muchen Li, Danica J. Sutherland, Xiaoxiao Li, and Christos Thrampoulidis. On the effect of negative gradient in group relative deep reinforcement optimization. arXiv preprint arXiv:2505.18830, 2025. URL `https://arxiv.org/abs/2505.18830`.

[7] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward rAnked FineTuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. URL `https://arxiv.org/abs/2304.06767`.

[8] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Model alignment as prospect theoretic optimization. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 12634–12651. PMLR, 2024.

[9] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.

[10] Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. Stream of search (SoS): Learning to search in language. In *First Conference on Language Modeling (COLM)*, 2024.

[11] Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized Markov decision processes. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 2160–2169. PMLR, 2019. URL `https://proceedings.mlr.press/v97/geist19a.html`.

[12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

[13] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018. URL `https://proceedings.mlr.press/v80/haarnoja18b.html`.

[14] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[15] Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11170–11189, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[16] Ronald A. Howard and James E. Matheson. Risk-sensitive Markov decision processes. *Management Science*, 18(7):356–369, 1972.

[17] Guanhua Huang, Tingqiang Xu, Mingze Wang, Qi Yi, Xue Gong, Siheng Li, Ruibin Xiong, Kejiao Li, Yuhao Jiang, and Bo Zhou. Low-probability tokens sustain exploration in reinforcement learning with verifiable reward. arXiv preprint arXiv:2510.03222, 2025. URL `https://arxiv.org/abs/2510.03222`.

[18] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 21314–21328, 2022.

[19] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018. URL `https://arxiv.org/abs/1805.00909`.

[20] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 3843–3857, 2022.

[21] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[22] Zhihang Lin, Mingbao Lin, Yuan Xie, and Rongrong Ji. CPPO: Accelerating the training of group relative policy optimization-based reasoning models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL `https://arxiv.org/abs/2503.22342`.

[23] Tianqi Liu, Zhen Qin, Junru Wu, Jiaming Shen, Misha Khalman, Rishabh Joshi, Yao Zhao, Mohammad Saleh, Simon Baumgartner, Jialu Liu, Peter J. Liu, and Xuanhui Wang. LiPO: Listwise preference optimization through learning-to-rank. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2404–2420, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.

[24] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL `https://arxiv.org/abs/2503.20783`.

[25] Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[26] Oliver Mihatsch and Ralph Neuneier. Risk-sensitive reinforcement learning. *Machine Learning*, 49(2):267–290, 2002.

[27] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016. URL `https://arxiv.org/abs/1602.01783`.

[28] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

[29] Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. TinyZero. `https://github.com/Jiayi-Pan/TinyZero`, 2025. Accessed 2026-07-12.

[30] Jaesung R. Park, Junsu Kim, Gyeongman Kim, Jinyoung Jo, Sean Choi, Jaewoong Cho, and Ernest K. Ryu. Clip-Low increases entropy and Clip-High decreases entropy in reinforcement learning of large language models. arXiv preprint arXiv:2509.26114, 2025. URL `https://arxiv.org/abs/2509.26114`.

[31] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. arXiv preprint arXiv:1910.00177, 2019. URL `https://arxiv.org/abs/1910.00177`.

[32] Jan Peters and Stefan Schaal. Reinforcement learning by reward-weighted regression for operational space control. In *International Conference on Machine Learning (ICML)*, pages 745–750, 2007.

[33] Jan Peters, Katharina M"ulling, and Yasemin Altun. Relative entropy policy search. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI)*, pages 1607–1612. AAAI Press, 2010. doi: 10.1609/aaai.v24i1.7727.

[34] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[35] John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897. PMLR, 2015. URL `https://proceedings.mlr.press/v37/schulman15.html`.

[36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL `https://arxiv.org/abs/1707.06347`.

[37] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[38] Feifan Song, Bowen Yu, Minghao Li, Haiyang Yu, Fei Huang, Yongbin Li, and Houfeng Wang. Preference ranking optimization for human alignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18990–18998, 2024.

[39] H. Francis Song, Abbas Abdolmaleki, Jost Tobias Springenberg, Aidan Clark, Hubert Soyer, Jack W. Rae, Seb Noury, Arun Ahuja, Siqi Liu, Dhruva Tirumala, Nicolas Heess, Dan Belov, Martin Riedmiller, and Matthew M. Botvinick. V-MPO: On-policy maximum a posteriori policy optimization for discrete and continuous control. In *International Conference on Learning Representations (ICLR)*, 2020. URL `https://openreview.net/forum?id=SylOlp4FvH`.

[40] Emanuel Todorov. Linearly-solvable Markov decision problems. In *Advances in Neural Information Processing Systems 19 (NeurIPS)*, volume 19, pages 1369–1376. MIT Press, 2006. URL `https://proceedings.neurips.cc/paper_files/paper/2006/hash/d806ca13ca3449af72a1ea5aedbed26a-Abstract.html`.

[41] Manan Tomar, Lior Shani, Yonathan Efroni, and Mohammad Ghavamzadeh. Mirror descent policy optimization. In *International Conference on Learning Representations (ICLR)*, 2022. URL `https://arxiv.org/abs/2005.09814`.

[42] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748, 2018. URL `https://arxiv.org/abs/1807.03748`.

[43] Nino Vieillard, Tadashi Kozuno, Bruno Scherrer, Olivier Pietquin, Rémi Munos, and Matthieu Geist. Leverage the average: An analysis of KL regularization in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL `https://proceedings.neurips.cc/paper/2020/hash/8e2c381d4dd04f1c55093f22c59c3a08-Abstract.html`.

[44] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL `https://openreview.net/forum?id=1PL1NIMMrw`.

[45] Muning Wen, Junwei Liao, Cheng Deng, Jun Wang, Weinan Zhang, and Ying Wen. Entropy-regularized token-level policy optimization for language agent reinforcement. arXiv preprint arXiv:2402.06700, 2024. URL `https://arxiv.org/abs/2402.06700`.

[46] Ronald J. Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

[47] Wenhao Zhan, Shicong Cen, Baihe Huang, Yuxin Chen, Jason D. Lee, and Yuejie Chi. Policy mirror descent for regularized reinforcement learning: A generalized framework with linear convergence. *SIAM Journal on Optimization*, 33(2):1061–1091, 2023. doi: 10.1137/21M1456789.

[48] Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The surprising effectiveness of negative reinforcement in LLM reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL `https://arxiv.org/abs/2506.01347`.

[49] Brian D. Ziebart, Andrew L. Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence (AAAI)*, pages 1433–1438. AAAI Press, 2008. URL `https://cdn.aaai.org/AAAI/2008/AAAI08-227.pdf`.

A Code-Faithful Objective Definitions

This appendix gives the exact training objectives used in the experimental quartet. G is the number of sampled completions per prompt, $\mathcal{A}$ the set of prompts in the batch, $N = |\mathcal{A}|\,G$ the number of sampled completions in the batch, and $T_{\max}$ the configured maximum completion length. All four methods share Appendices A.1 and A.2 and differ only in Appendices A.3–A.6.

A.1 Common rollout setup

For each prompt x the rollout policy μ samples completions $\{y_{x,1}, \ldots, y_{x,G}\}$ with scalar rewards $r_{x,i} \in \{0,1\}$ from final-answer verification. Define the group mean and the centered group-relative advantage

$$\bar{r}_x = \frac{1}{G}\sum_{i=1}^{G} r_{x,i}, \qquad A_{x,i} = r_{x,i} - \bar{r}_x \;\in\; \Big[-\big(1-\tfrac{1}{G}\big),\; 1-\tfrac{1}{G}\Big].$$

The implementation also supports standard-deviation scaling, $A_{x,i} = (r_{x,i} - \bar{r}_x)/(\mathrm{Std}(\{r_{x,j}\}_j) + 10^{-8})$, but this is disabled in every released recipe (the `critic_type=drgrpo` setting selects the unscaled advantage and the constant-normalizer aggregator below), consistent with the Dr.GRPO correction [24].

Let $m_{x,i,t} \in \{0,1\}$ be the completion-token mask, $\log \mu_{x,i,t}$ the rollout log-probability, and $\log \pi_{\theta,x,i,t}$ the current-policy log-probability. The tokenwise importance ratio and its clipped version are

$$c_{x,i,t} = \exp\!\big(\log \pi_{\theta,x,i,t} - \log \mu_{x,i,t}\big), \qquad \tilde{c}_{x,i,t} = \mathrm{clip}\big(c_{x,i,t},\, 1-\epsilon_{\mathrm{low}},\, 1+\epsilon_{\mathrm{high}}\big).$$

An optional tokenwise reverse-KL penalty to a frozen reference policy is available:

$$D_{x,i,t} = \exp(\delta_{x,i,t}) - \delta_{x,i,t} - 1, \qquad \delta_{x,i,t} = \mathrm{clip}\big(\log \pi_{\mathrm{ref},x,i,t} - \log \pi_{\theta,x,i,t},\, -40,\, 40\big),$$

weighted by $\beta \geq 0$. This is the *only* mechanism in the paper that constrains π_θ toward π_{ref}; see the warning in Appendix B.1.

A.2 Per-candidate utility and the shared backend

All four objectives are built from a single per-candidate quantity, the *normalized Dr.GRPO utility*

$$\bar{U}_{x,i}(\theta) := \frac{1}{T_{\max}} \sum_t m_{x,i,t}\, \min\!\big(c_{x,i,t} A_{x,i},\; \tilde{c}_{x,i,t} A_{x,i}\big), \tag{9}$$

together with the tokenwise penalty term $P_{x,i}(\theta) := \frac{1}{T_{\max}} \sum_t m_{x,i,t}\, D_{x,i,t}$. The constant $T_{\max}$ normalizer — rather than the per-completion length $|y_{x,i}|$ — is exactly Dr.GRPO's length-bias correction.

Every method in the quartet then has the form

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{A}|}\sum_{x\in\mathcal{A}}\sum_{i=1}^{G}\Big[-\underbrace{a_{x,i}}_{\text{aggregation weight}}\bar{U}_{x,i}(\theta) \;+\; \tfrac{\beta}{G}\,P_{x,i}(\theta)\Big] \;+\; \underbrace{R(\theta)}_{\text{optional extra regularizer}}, \tag{10}$$

and the four methods are distinguished entirely by $(a_{x,i}, R)$. This is the axis the paper studies. Table 2 is the whole quartet in one place.

A.3 Dr.GRPO

The baseline sets $a_{x,i} = 1/G$:

$$\mathcal{L}_{\mathrm{Dr.GRPO}}(\theta) = -\frac{1}{|\mathcal{A}|\,G}\sum_{x\in\mathcal{A}}\sum_{i=1}^{G}\bar{U}_{x,i}(\theta) + \frac{\beta}{|\mathcal{A}|\,G}\sum_{x,i}P_{x,i}(\theta) \;=\; \frac{1}{N\,T_{\max}}\sum_{x,i,t} m_{x,i,t}\Big[-\min\!\big(c_{x,i,t}A_{x,i}, \tilde{c}_{x,i,t}A_{x,i}\big)+\beta D_{x,i,t}$$

which is the standard implementation-faithful form. The equality is immediate from $N = |\mathcal{A}|G$ and (9). *The group-relative signal lives entirely in $A_{x,i}$; the aggregation across the group is a plain mean.*

Method	Aggregation weight $a_{x,i}$	Extra regularizer $R(\theta)$ / notes
Dr.GRPO	$1/G$ (uniform)	none. Risk-neutral aggregation.
SEED-Dr.GRPO	$s_x/G,\ s_x = \left(1+\alpha_{\text{eff}} H_x^{\text{sem}}\right)^{-1}$	none. Uniform within the group; the whole *prompt* is rescaled.
Token-MaxEnt Dr.GRPO	$1/G$ (uniform)	$-\frac{\alpha}{\lvert\mathcal{A}\rvert\, G\, T_{\max}} \sum_{x,i,t} m_{x,i,t} \mathcal{H}_{x,i,t}(\pi_\theta)$
xDr.GRPO	$\mathrm{sg}\left[\dfrac{\exp(\bar{U}_{x,i}/\kappa)\,(\pi_{x,i}^{\text{ref}})^{\lambda/\kappa}}{\sum_j \exp(\bar{U}_{x,j}/\kappa)\,(\pi_{x,j}^{\text{ref}})^{\lambda/\kappa}}\right]$	none. $\kappa := \tau + \lambda$. Recovers Dr.GRPO exactly as $\tau \to \infty$.

Table 2: **The quartet is one line of code.** All methods instantiate (10) with the same per-candidate utility (9); they differ only in the aggregation weight $a_{x,i}$ and an optional extra regularizer. $\mathrm{sg}[\cdot]$ denotes `stop_gradient` (see Remark B.3).

A.4 SEED-Dr.GRPO

SEED preserves the clipped surrogate but rescales each prompt's advantages by semantic predictive entropy. Let $c(i)$ be the semantic answer cluster of completion i and

$$\ell_{x,i} = \frac{1}{|y_{x,i}|} \sum_t m_{x,i,t} \log \mu_{x,i,t}, \qquad p_{x,c} = \frac{\sum_{i:\, c(i)=c} \exp(\ell_{x,i})}{\sum_{j=1}^{G} \exp(\ell_{x,j})}, \qquad H_x^{\text{sem}} = -\sum_c p_{x,c} \log p_{x,c}.$$

We length-normalize $\ell_{x,i}$ (see Appendix B.1 for why), and evaluate it at the rollout policy μ, matching the frozen-weight convention of Remark A.1 (at the single update of the released one-epoch recipe, $\mu = \pi_\theta$). Semantic clusters are the canonical answer modes of the verifier — countdown expressions reduce to canonical ASTs, colorings to completion strings — and completions with no extractable answer form singleton clusters. The prompt-level scaling is $s_x = (1 + \alpha_{\text{eff}} H_x^{\text{sem}})^{-1}$ with $\alpha_{\text{eff}} = \alpha/\log G$, giving $a_{x,i} = s_x/G$. Note that s_x does not depend on i: SEED reweights *prompts*, and remains uniform *within* the group.

Fidelity to, and one departure from, the released SEED-GRPO. The functional form above matches the released SEED-GRPO implementation, which likewise scales advantages by $(1 + \alpha' H^{\text{sem}})^{-1}$ with length-normalized sequence likelihoods and the Shannon entropy of the likelihood-weighted cluster masses (rather than the Monte-Carlo estimator $-\frac{1}{K}\sum_k \log p_{c(k)}$ written in that paper), and which also assigns unparseable completions to singleton clusters [3]. The one deliberate departure is the sensitivity: the released SEED-GRPO default corresponds to $\alpha \approx 0.04$ in our parameterization, which caps the prompt rescaling at about 4% even at maximal semantic entropy — a perturbation of Dr.GRPO far below this study's minimum detectable effect. We instead run the arm at $\alpha = 1$, which reaches $s_x = \frac{1}{2}$ at maximal entropy, so that the prompt-level mechanism is actually testable at our sample sizes; results for this arm should be read as testing the *mechanism at a visible dose*, not the original tuning.

A.5 Token-MaxEnt Dr.GRPO

Token-MaxEnt keeps $a_{x,i} = 1/G$ and adds an exact next-token entropy bonus with

$$\mathcal{H}_{x,i,t}(\pi_\theta) = -\sum_v \pi_\theta(v \mid x, y_{x,i,<t}) \log \pi_\theta(v \mid x, y_{x,i,<t}), \qquad R(\theta) = -\frac{\alpha}{|\mathcal{A}|\, G\, T_{\max}} \sum_{x,i,t} m_{x,i,t}\, \mathcal{H}_{x,i,t}(\pi_\theta).$$

This is the classic loss-side policy-entropy bonus [46, 27] placed on the Dr.GRPO backbone, with the entropy computed exactly over the vocabulary (no sampling estimate) and aggregated with the same constant $T_{\max}$ normalizer as the surrogate. Entropy here acts on the next-token distribution, not on the sampled group.

A.6 xDr.GRPO

xDr.GRPO changes only $a_{x,i}$. Let $\pi_x^{\text{ref}} \in \Delta^G$ be the reference distribution over the sampled candidates (Appendix B.1), let $\tau > 0$, $\lambda \geq 0$, and $\kappa := \tau + \lambda$. Then

$$w_{x,i}^{\star}(\theta) = \frac{\exp\big(\bar{U}_{x,i}(\theta)/\kappa\big)\,\big(\pi_{x,i}^{\text{ref}}\big)^{\lambda/\kappa}}{\sum_{j=1}^{G} \exp\big(\bar{U}_{x,j}(\theta)/\kappa\big)\,\big(\pi_{x,j}^{\text{ref}}\big)^{\lambda/\kappa}}, \qquad a_{x,i} := \mathrm{sg}\big[w_{x,i}^{\star}(\theta)\big], \tag{11}$$

so that

$$\mathcal{L}_{\text{xDr.GRPO}}(\theta) = -\frac{1}{|\mathcal{A}|}\sum_{x\in\mathcal{A}}\sum_{i=1}^{G}\operatorname{sg}\big[w^{\star}_{x,i}\big]\,\bar{U}_{x,i}(\theta)\;+\;\frac{\beta}{|\mathcal{A}|\,G}\sum_{x,i}P_{x,i}(\theta). \tag{12}$$

Equation (12) is the negative of the log-partition objective $\mathcal{J}_{\text{xDr.GRPO}}$ of Section 3; the identification and the necessity of the sg are Proposition B.2 and Remark B.3.

Diff against the baseline. The entire implementation delta is:

```
# Utilities at the rollout policy (importance ratio 1): the clipped
# surrogate reduces to U_i = A_i * T_i / T_max per candidate.
U = adv * response_token_counts / T_max          # [num_prompts, G]

# --- Dr.GRPO (tau = inf) ---
# the reweighting below is skipped entirely; the baseline loss
# aggregation runs unchanged.

# --- xDr.GRPO (finite tau) ---
w = G * torch.softmax(U / tau, dim=-1).detach()  # <-- detach is required
loss = -(w * U_theta * loss_masks).mean()        # per-row reweighting
```

Here U_θ denotes the tokenwise clipped surrogate recomputed at the current parameters inside the update loop; only its per-row aggregation weight changes. Rollouts, rewards, clipping, masking, optimizer, and the tokenwise KL term are untouched.

Remark A.1 (Weights are frozen at the rollout policy). The released implementation computes $w^{\star}_x$ once per rollout batch from the utilities evaluated at the rollout policy μ — where the importance ratio is 1 and $\bar{U}_{x,i} = A_{x,i}\,T_{x,i}/T_{\max}$ exactly — and holds them fixed for the update, exactly as the advantages themselves are. Under the released recipe (one optimization epoch, weights consumed at the first gradient step where $\pi_\theta = \mu$) this coincides with $w^{\star}(\theta)$; in multi-epoch configurations the two diverge and $w^{\star}(\theta)$ would need recomputing per step. Setting $\tau = \infty$ disables the reweighting entirely, leaving the baseline aggregation *bit-for-bit* unchanged; the $\tau = \infty$ arm of the sweep in Appendix B.5 is run through the same launcher with the reweighting skipped. The reference tilt ($\lambda > 0$, Appendix B.1) is presented for generality; the released implementation fixes $\lambda = 0$.

Degenerate groups. If all rewards in a group are equal then $A_{x,i} = 0$ for all i, hence $\bar{U}_{x,i} = 0$, $w^{\star}_x$ is uniform (or purely reference-tilted), and the gradient of (12) vanishes. Uninformative prompts drop out automatically, matching Dr.GRPO. No prompt-skipping heuristic is used.

A.7 Shared recipe simplifications

All matched recipes use

$$G = 16, \qquad \texttt{critic\_type} = \texttt{drgrpo}, \qquad \beta = 0, \qquad \lambda = 0,$$

with a learning rate of 2×10^{-7}, a training budget of 9,216 prompt consumptions, one optimization epoch per rollout batch, and sampling temperature 1. The group size and learning rate were calibrated on the Dr.GRPO baseline alone, before any cross-method comparison: at $G = 8$ most groups contain no correct sample at this model scale (starving the group-relative signal entirely), and at learning rates $\geq 10^{-6}$ the policy's training success rate collapses toward zero within a run, independent of method. The remaining pre-specified constants are the Token-MaxEnt coefficient $\alpha = 0.01$, the SEED sensitivity $\alpha = 1$, and the primary xDr.GRPO temperature $\tau = 0.5$ (the remaining grid points are reported as exploratory, with Holm-adjusted p-values). With $\beta = 0$ the tokenwise KL term vanishes; with $\lambda = 0$ the reference tilt vanishes and $\kappa = \tau$. The comparison therefore isolates a single axis: the aggregation weight $a_{x,i}$ (uniform for Dr.GRPO, prompt-rescaled uniform for SEED, uniform-plus-token-entropy for Token-MaxEnt, tempered-softmax for xDr.GRPO).

B Theory

Throughout, fix a prompt x and abbreviate $\bar{U}_i := \bar{U}_{x,i}(\theta)$ from (9), $\pi_i := \pi^{\text{ref}}_{x,i}$, $w_i := w_{x,i}$, and $\kappa := \tau + \lambda > 0$. We assume $\pi_i > 0$ for all i and that $\theta \mapsto \bar{U}_i(\theta)$ is differentiable at θ.

B.1 The reference tilt is an aggregation prior, not a policy trust region

The reference distribution over the sampled candidate set is

$$\pi^{\mathrm{ref}}_{x,i} = \frac{\exp\big(\ell^{\mathrm{ref}}_{x,i}\big)}{\sum_j \exp\big(\ell^{\mathrm{ref}}_{x,j}\big)}, \qquad \ell^{\mathrm{ref}}_{x,i} = \frac{1}{|y_{x,i}|}\sum_t m_{x,i,t}\log \pi_{\mathrm{ref}}(y_{x,i,t} \mid x, y_{x,i,<t}).$$

Length normalization is not optional here. An unnormalized sequence log-likelihood decreases monotonically in completion length, so a softmax over raw log-likelihood sums places mass on short completions for reasons that have nothing to do with quality. Since Dr.GRPO's central correction is the removal of exactly this length bias, a candidate distribution built from raw sums would reintroduce it at the aggregation step. We therefore length-normalize ℓ^{ref}, and likewise $\ell_{x,i}$ in SEED (Appendix A.4).

λ is not β. The term $-\lambda \mathrm{KL}(w\|\pi^{\mathrm{ref}}_x)$ acts on the *aggregation weights* $w \in \Delta^G$. It does *not* bound $\mathrm{KL}(\pi_\theta\|\pi_{\mathrm{ref}})$ and is *not* a trust region on the policy: one can drive π_θ arbitrarily far from π_{ref} while w stays pinned to π^{ref}_x. Trust-region control over the policy, where wanted, is supplied separately by the tokenwise penalty $\beta D_{x,i,t}$ of Appendix A.1. We therefore call λ a *reference tilt*: it biases aggregation toward candidates the base model already finds plausible, damping the influence of low-likelihood off-policy outliers on the update. It is a prior over *whom to learn from*, not a leash on *where to go*.

B.2 Closed-form aggregation weights

Proposition B.1 (Closed form)**.** *The map $w \mapsto \langle w, \bar{U}\rangle + \tau H(w) - \lambda \mathrm{KL}(w\|\pi^{\mathrm{ref}}_x)$ is strictly concave on Δ^G and attains its unique maximum at*

$$w^\star_i = \frac{\exp(\bar{U}_i/\kappa)\,\pi_i^{\lambda/\kappa}}{\sum_{j=1}^G \exp(\bar{U}_j/\kappa)\,\pi_j^{\lambda/\kappa}}.$$

Proof. Collecting the $w_i \log w_i$ terms,

$$\langle w, \bar{U}\rangle + \tau H(w) - \lambda \mathrm{KL}(w\|\pi^{\mathrm{ref}}_x) = \sum_i w_i \bar{U}_i - \kappa \sum_i w_i \log w_i + \lambda \sum_i w_i \log \pi_i.$$

The map $w \mapsto -\sum_i w_i \log w_i$ is strictly concave on the simplex and $\kappa > 0$; the remaining terms are linear in w. The objective is therefore strictly concave, so any stationary point is the unique maximizer. Introducing a multiplier η for $\sum_i w_i = 1$, stationarity reads $\bar{U}_i - \kappa(\log w_i + 1) + \lambda \log \pi_i + \eta = 0$, whence $w_i \propto \exp(\bar{U}_i/\kappa)\,\pi_i^{\lambda/\kappa}$ (the factor $\exp((\eta-\kappa)/\kappa)$ is common to all coordinates). Normalizing gives the claim. □

Proposition B.2 (Log-partition form and gradient)**.** *Let $Z := \sum_j \exp(\bar{U}_j/\kappa)\,\pi_j^{\lambda/\kappa}$ and let $\mathcal{G}_x(\theta)$ denote the optimal value of the objective in Proposition B.1. Then*

$$\mathcal{G}_x(\theta) = \kappa \log Z, \qquad \textit{and} \qquad \nabla_\theta \mathcal{G}_x(\theta) = \sum_{i=1}^G w^\star_i(\theta)\,\nabla_\theta \bar{U}_i(\theta).$$

Proof. Substituting $w^\star$ and using $\log w^\star_i = \bar{U}_i/\kappa + (\lambda/\kappa)\log \pi_i - \log Z$ into the objective, the $\bar{U}$, entropy, and $\log \pi$ terms cancel pairwise and leave $\mathcal{G}_x = \kappa \log Z$. For the gradient, π^{ref} does not depend on θ, so

$$\nabla_\theta \mathcal{G}_x = \kappa Z^{-1}\nabla_\theta Z = \kappa Z^{-1}\sum_i \exp(\bar{U}_i/\kappa)\,\pi_i^{\lambda/\kappa}\cdot \kappa^{-1}\nabla_\theta \bar{U}_i = \sum_i w^\star_i\,\nabla_\theta \bar{U}_i. \qquad \square$$

Remark B.3 (Why the weights must be detached). Proposition B.2 is an envelope (Danskin) statement: although $w^\star$ depends on θ, that dependence contributes nothing to $\nabla_\theta \mathcal{G}_x$, because $w^\star$ is a stationary point of the inner problem. Consequently the correct implementation of (12) treats $w^\star$ as a constant. Backpropagating through the softmax optimizes a different — and uncharacterized — objective, and is the single easiest way to get this method wrong.

B.3 Dr.GRPO is the risk-neutral limit of the family

Proposition B.4 (Variance expansion). *Let $\lambda = 0$, and set $\mu_x := \frac{1}{G}\sum_i \bar{U}_i$ and $\sigma_x^2 := \frac{1}{G}\sum_i(\bar{U}_i - \mu_x)^2$. Then, as $\tau \to \infty$,*

$$\mathcal{G}_x(\theta) = \tau \log G \;+\; \underbrace{\mu_x(\theta)}_{\text{Dr.GRPO}} \;+\; \underbrace{\frac{\sigma_x^2(\theta)}{2\tau}}_{\text{exploration bonus}} \;+ O(\tau^{-2}).$$

Proof. Put $\epsilon := 1/\tau$. Then $\mathcal{G}_x = \epsilon^{-1}\log\sum_i e^{\epsilon \bar{U}_i} = \epsilon^{-1}\log G + \epsilon^{-1}\log\big(\frac{1}{G}\sum_i e^{\epsilon\bar{U}_i}\big)$. The second logarithm is the cumulant generating function of the empirical distribution of $\{\bar{U}_i\}_{i=1}^G$ evaluated at ϵ; since the $\bar{U}_i$ are uniformly bounded (Appendix B.5) it is analytic at 0 with expansion $\epsilon\mu_x + \frac{\epsilon^2}{2}\sigma_x^2 + O(\epsilon^3)$. Multiplying by $\epsilon^{-1} = \tau$ gives the claim. □

Proposition B.5 (Gradient convergence to Dr.GRPO). *Let $\lambda = 0$, $M_x := \max_i |\bar{U}_i(\theta)|$, and $L_x := \max_i \|\nabla_\theta \bar{U}_i(\theta)\|$. Then*

$$\Big\| \nabla_\theta \mathcal{G}_x(\theta) - \nabla_\theta \mathcal{J}_{\text{Dr.GRPO},x}(\theta) \Big\| \;\le\; \big(e^{2M_x/\tau} - 1\big)\, L_x \;=\; O(1/\tau),$$

where $\mathcal{J}_{\text{Dr.GRPO},x}(\theta) = \frac{1}{G}\sum_i \bar{U}_i(\theta)$ is the prompt-x contribution to the Dr.GRPO objective (Appendix A.3).

Proof. By Proposition B.2, $\nabla\mathcal{G}_x - \nabla\mathcal{J}_{\text{Dr.GRPO},x} = \sum_i \big(w_i^\star - \frac{1}{G}\big)\nabla\bar{U}_i$, so by the triangle inequality the norm is at most $\|w^\star - \mathbf{u}\|_1 L_x$ with $\mathbf{u}$ uniform. Apply Proposition B.8 to the pair $(\bar{U}, 0)$, noting that with $\lambda = 0$ the weights induced by the all-zero utility vector are exactly $\mathbf{u}$, and that $\|\bar{U} - 0\|_\infty = M_x$. This gives $\big|\log(w_i^\star/\frac{1}{G})\big| \le 2M_x/\tau$ for every i, hence $\big|w_i^\star - \frac{1}{G}\big| \le \frac{1}{G}\big(e^{2M_x/\tau} - 1\big)$ and, summing, $\|w^\star - \mathbf{u}\|_1 \le e^{2M_x/\tau} - 1$. Finally $e^{2M_x/\tau} - 1 = O(1/\tau)$ for bounded M_x. □

Corollary B.6 (Optimism). *For every $\tau > 0$ with $\lambda = 0$,*

$$\mathcal{G}_x(\theta) - \tau\log G \;\ge\; \mathcal{J}_{\text{Dr.GRPO},x}(\theta),$$

with equality if and only if $\bar{U}_1 = \cdots = \bar{U}_G$.

Proof. $\tau\log\big(\frac{1}{G}\sum_i e^{\bar{U}_i/\tau}\big) \ge \frac{1}{G}\sum_i \bar{U}_i$ by Jensen's inequality applied to the strictly convex map $u \mapsto e^{u/\tau}$; equality holds iff $e^{\bar{U}_i/\tau}$ is constant in i, i.e. iff the $\bar{U}_i$ coincide. □

Remark B.7 (What the theory actually says). Read together, Propositions B.4–B.5 and Corollary B.6 give the paper's claim its precise form. *(i)* Dr.GRPO is not "xDr.GRPO without exploration": it is xDr.GRPO at $\tau = \infty$, the risk-neutral corner of a one-parameter family. The uniform aggregation in Dr.GRPO is an unexamined default, not a derived quantity. *(ii)* Finite τ replaces the arithmetic mean of candidate utilities by their tempered log-sum-exp — the *entropic risk measure* — which by Corollary B.6 upper-bounds the Dr.GRPO objective by exactly the within-group *dispersion* of candidate utilities. Maximizing that bound rewards groups whose candidates still disagree. This is optimism in the face of uncertainty, applied to the object GRPO uses for credit assignment. *(iii)* Mechanically, $w_{x,i}^\star \propto \exp(\bar{U}_{x,i}/\tau)$ and $\bar{U}_{x,i}$ is increasing in $A_{x,i}$, so negative-advantage candidates receive exponentially attenuated gradients. The theory establishes the attenuation, not its downstream effect on policy entropy: the token-level literature attributes entropy collapse variously to high-covariance positive-advantage updates [5] — with sequence-level negative reinforcement alone even *preserving* entropy [48] — and to negative gradients on low-probability tokens [30, 17, 6]. Whether candidate-level attenuation sustains a higher effective rollout count and a lower incorrect-mass share is what Figure 1 is designed to measure.

B.4 Stability under utility perturbation

The utilities $\bar{U}_i$ are computed from noisy verifier rewards (and, in multi-epoch configurations, drifting importance ratios), so the aggregation weights must not be brittle in them. They are not: the entropy weight controls the Lipschitz constant directly.

Proposition B.8 (Coordinatewise stability). *Let $\bar{U}, \widetilde{\bar{U}} \in \mathbb{R}^G$ with $\|\widetilde{\bar{U}} - \bar{U}\|_\infty \le \varepsilon$, and let $w^\star, \widetilde{w}^\star$ be the corresponding weights from Proposition B.1 under the same $\tau, \lambda, \pi_x^{\mathrm{ref}}$. Then for every i,*

$$\left|\log \tfrac{\widetilde{w}_i^\star}{w_i^\star}\right| \;\le\; \frac{2\varepsilon}{\kappa}, \qquad \textit{and} \qquad \left|\widetilde{\mathcal{G}}_x - \mathcal{G}_x\right| \;\le\; \varepsilon.$$

Proof. Write $Z = \sum_j \exp(\bar{U}_j/\kappa)\, \pi_j^{\lambda/\kappa}$ and $\widetilde{Z}$ for its counterpart under $\widetilde{\bar{U}}$. Since $|\widetilde{\bar{U}}_j - \bar{U}_j| \le \varepsilon$ for every j, each summand of $\widetilde{Z}$ is within a factor $e^{\pm\varepsilon/\kappa}$ of the corresponding summand of Z, hence $e^{-\varepsilon/\kappa} \le \widetilde{Z}/Z \le e^{\varepsilon/\kappa}$, i.e. $|\log(\widetilde{Z}/Z)| \le \varepsilon/\kappa$. For the weights,

$$\log \frac{\widetilde{w}_i^\star}{w_i^\star} = \frac{\widetilde{\bar{U}}_i - \bar{U}_i}{\kappa} - \log \frac{\widetilde{Z}}{Z}, \qquad \text{so} \qquad \left|\log \frac{\widetilde{w}_i^\star}{w_i^\star}\right| \le \frac{\varepsilon}{\kappa} + \frac{\varepsilon}{\kappa} = \frac{2\varepsilon}{\kappa}.$$

For the objective values, Proposition B.2 gives $\widetilde{\mathcal{G}}_x - \mathcal{G}_x = \kappa \log(\widetilde{Z}/Z)$, whose absolute value is at most $\kappa \cdot \varepsilon/\kappa = \varepsilon$. $\square$

B.5 Boundedness of the utilities and the practical range of τ

Boundedness. The utilities that enter the aggregation weights are evaluated at the rollout policy (Remark A.1), where the importance ratio is exactly 1 and every summand of (9) equals $A_{x,i}$ on active positions. Since the mask has at most $T_{\max}$ active positions and $|A_{x,i}| \le 1 - \frac{1}{G}$,

$$M_x \;=\; \max_i \left|\bar{U}_{x,i}\right| \;=\; \max_i \,|A_{x,i}|\, \frac{T_{x,i}}{T_{\max}} \;\le\; 1 - \frac{1}{G}$$

exactly, with no clipping argument required. (For the general current-θ surrogate the positive branch is still bounded by $(1+\epsilon_{\text{high}})|A_{x,i}|$, but the unclipped negative branch is unbounded in principle; the released weights never evaluate it off-policy.) This uniform boundedness is what Proposition B.4 uses, and it justifies treating $[-1, 1]$ as the utility scale when choosing τ.

Practical range of τ. The weights depend on the utilities only through $\bar{U}_{x,i}/\kappa$, so the meaningful scale for τ is the within-group utility spread, which for binary rewards and $G = 16$ is of order 10^{-1} after the $T_{\max}$ normalization. The sweep uses the grid $\tau \in \{0.05, 0.1, 0.25, 0.5, 1, 2, \infty\}$ with $\lambda = 0$ (Table 3), where the $\tau = \infty$ arm disables the reweighting entirely and runs the identical baseline code path (Remark A.1), reproducing Dr.GRPO bit-for-bit. Values of τ far below the utility spread approach the best-of-G corner and inherit its instability (Proposition B.8); values far above it are indistinguishable from Dr.GRPO by Proposition B.5. Reported τ values should be read relative to the utility scale $M_x \le 1$ above.

C Implementation and Evaluation Details

The design principle is parity: all compared methods use the same data, model, prompt template, rollout budget, reward extraction, and evaluation harness. Only the aggregation weight $a_{x,i}$ of (10) differs.

Objective-specific difference. Dr.GRPO aggregates per-candidate utilities uniformly within each prompt group. xDr.GRPO aggregates the *same* utilities with the tempered softmax weights of (11), computed at the rollout policy and frozen per rollout batch (Remark A.1). SEED-Dr.GRPO rescales whole prompts by semantic entropy; Token-MaxEnt Dr.GRPO adds a next-token entropy bonus. Nothing else changes.

Diagnostics. The candidate-level diagnostics are defined for every method through the realized aggregation weights $a_{x,i}$ of (10): the effective number of active rollouts $\exp\big(H(a_x)\big)$, with a_x normalized within the prompt group over the rows that train, and the share of aggregation mass falling on incorrect rollouts $\sum_{i:\, r_{x,i}=0} a_{x,i}$. Because Dr.GRPO also has an explicit $a_{x,i}$ under (10) (namely $1/G$), the training loop computes both quantities identically for every arm at every step — uniform weights for Dr.GRPO and Token-MaxEnt, the prompt-rescaled uniform weights for SEED (whose within-group distribution is uniform, so its curves coincide with the baseline's by construction), and the tempered softmax weights for xDr.GRPO — and logs them under a shared key, so the curves are directly comparable. Figure 1 plots exactly these quantities.

Component	Setting
Base model	Qwen2.5-0.5B-Instruct.
Training data	Synthetic exact multi-answer Countdown arithmetic (384 train / 128 eval prompts, 2–8 modes) and graph-coloring completion (192 / 96), regenerated deterministically from fixed seeds.
Prompting	Single-boxed-answer instruction prompt per domain, identical in training and evaluation.
Rollouts	$G = 16$ completions per prompt for every arm; sampling temperature 1.0, top-p 1.0; prompt length ≤ 256 tokens; completion budget $T_{\max} = 192$ tokens; one prompt group per learner step.
Optimization	Constant learning rate 2×10^{-7} (Adam), one optimization epoch per rollout batch (`num_ppo_epochs=1`), PPO clip range $\epsilon = 0.2$ (symmetric), gradient-norm clip 1.0, bfloat16; training budget $9{,}216$ prompt consumptions per run; `critic_type=drgrpo` (constant $T_{\max}$ normalizer, mean-only advantages); $\beta = 0$, no KL pseudo-reward, so no reference model enters training.
Main paired methods	Dr.GRPO and xDr.GRPO.
Additional baselines	SEED-Dr.GRPO and Token-MaxEnt Dr.GRPO, same backbone (Appendix A.2).
Arm-specific settings	xDr.GRPO: $\tau \in \{0.05, 0.1, 0.25, 0.5, 1, 2, \infty\}$ with $\tau = 0.5$ pre-specified as primary (remaining grid points exploratory, Holm-adjusted); $\lambda = 0$; $\tau = \infty$ *is* the Dr.GRPO arm. SEED-Dr.GRPO: $\alpha = 1$ (Appendix A.4). Token-MaxEnt: $\alpha = 0.01$.
Training seeds	Three independent fine-tuning seeds (43, 44, 45) per method and per τ.
Evaluation seeds	Three stochastic evaluation seeds per trained checkpoint; $K = 8$ sampled completions per prompt at temperature 1.0, plus one greedy pass.
Evaluation splits	Held-out multi-answer split per domain; optional single-mode control split.
Checkpoint selection	Final saved checkpoint of each run; rule fixed before any evaluation, not tuned on the evaluation splits.
Primary estimand	Coefficient on the method indicator in Equation 8.

Table 3: **Matched run design.** The repeated seed and task structure is part of the experimental design, not an after-the-fact aggregation choice. Because Dr.GRPO is the $\tau = \infty$ member of the xDr.GRPO family, the baseline and the treatment share one code path, which removes the most common source of unequal-tuning confounds.

Regression-ready logging. For each evaluation we log the method, τ, training seed, evaluation seed, task domain, prompt identifier, sampled-completion index, the normalized predicted answer and its canonical mode key, the prompt's number of valid modes, and the correctness indicator. This yields the observation-level table required by Equation 8. Aggregate accuracy and coverage tables are derived from the same records, so the descriptive and regression results are guaranteed to agree.

`[pending: run ops/plot_exploration_sidecar.py on a training round that wrote train_metrics.jsonl]`

Figure 1: **Candidate-level exploration diagnostics.** Effective number of active rollouts $\exp(H(a_x))$ (left; uniform aggregation sits at the number of valid rollouts per group) and incorrect-mass share (right) over training. Lines are means over the three training seeds, bands are seed min–max, and both quantities are computed from the realized aggregation weights logged identically for every method in the quartet.